

Working Notes X

PERSONAL WORKING DOCUMENT — NOT A REPORT FOR THE
PROFESSOR, NOT PAPER TEXT

Read-Out-Only Fine-Tuning on Three-Way Splits: Targeted vs. Full-Distribution

Edoardo Paccagnella

August 26, 2026

This document exists so Edoardo can read Claude’s analysis of one specific experiment before deciding whether/how it goes into the paper (`idea_paper_by_prof.tex`, §5.3 “Generalization to three paths, finetuning??”). It is not written for the professor and is not paper prose — do not copy sentences from here into the paper without rewriting them to the paper’s own conventions and evidentiary bar.

1 Where this comes from

Report IX, Thread A.3 (`report/9/transformer_for_graphs_9.tex`, and the mechanistic follow-up `stagewise_threeway.py`) tested the split-chains-only $n=46$ seed-1000 checkpoint on every three-way partition of the canvas. Two cells stood out on inspection of the five-stage similarity-geometry heatmaps:

- (15, 15, 16) — fully balanced. Every within-component block in the final-stage geometry is already visibly darker/more positive than the cross-component blocks (it *looks* like the model has separated the three components correctly), yet all three pairwise cuts fail and exact match is 0: the cross-component logits sit just *above* the model’s own decision threshold, not far below it.
- (7, 15, 24) — two of the three cuts succeed, `cut(2, 3)` (middle vs. largest) fails; the 24-node component also contains pairs beyond the doubled wall $2 \cdot 3^L=18$.

This document runs *two* fine-tuning experiments off the same starting checkpoint, asking two related but different questions.

- **Experiment 1 (targeted, §2.1–§2.3).** Does fine-tuning *only* the similarity read-out’s two global scalars (scale, bias) — everything else in the trunk frozen — on a stream containing *only these two named cells* fix them, without breaking the ordinary $K=2$ split-chains behaviour the checkpoint already has?
- **Experiment 2 (full distribution, §3.1–§3.3).** Does the same two-scalar fine-tuning, given exposure to *every* three-way split instead of just two cells, and a much longer budget, learn a genuinely *general* three-component rule rather than overfitting the two cells above?

Both are narrower/longer than Report IX’s own fine-tuning probe (`report/9/scratch_prof_experiments/§6`), which trained on a generic $K \in \{3, 4, 5, 6\}$ `path_union` stream for only 3,000 steps and left $K \geq 3$ exact match at 0 throughout. Fine-tuning only two scalars over a frozen trunk is cheap, so a much longer budget costs little in either case.

2 Experiment 1: targeted fine-tuning on the two named cells

2.1 Setup

Checkpoint. The unmodified split-chains-only, $n=46$, seed-1000, similarity read-out checkpoint from Report IX (`runs/report9/n46_train/n46_split_chains_roberta_similarity_lam0_seed1000/`) — the same one behind every Report IX §A.3 three-way figure.

Fine-tuning. Every parameter frozen except `sim_scale/sim_bias` (2 scalars out of $\sim 6.3\text{M}$). Training stream: uniformly, either the (15, 15, 16) or the (7, 15, 24) three-way split-chain graph (`generate_multi_path_split_graph`), freshly relabelled every sample. 50,000 steps, batch 500, AdamW lr 10^{-3} (`finetune_readout_threeway.py`).

What is measured. Before fine-tuning (step 0) and every 1,000 steps after: whole-graph exact match, per-component reach, the three pairwise cuts, and the overall predicted-positive rate, on both target cells (so the effect on the *held-out* cell is visible even when only the other one dominates gradient updates); for (7, 15, 24) specifically, reach inside the 24-node component split into three shortest-path-distance buckets — within-capacity ($d \leq 9$), between-the-walls ($9 < d \leq 18$), and beyond the doubled wall ($d > 18$) — the direct test of whether fine-tuning can push learning past the architectural bound, not just recalibrate a threshold within it. Separately, `eval_asym_chains.py` on the fine-tuned checkpoint reproduces the ordinary $K=2$ split-chains sweep (Report IX Table 3/§sec:planA), compared against the identical, already-committed sweep on the *untouched* checkpoint (`runs/report9/asym_chains_n46/n46_splitchains_seed1000/`), to check the two-scalar update did not cost anything in distribution.

2.2 Results

2.2.1 Does it learn the two target cells?

Figure 1 answers this only partially. On (15, 15, 16), `cut(2, 3)` climbs cleanly from 0.20 at step 0 to 1.000 by step $\sim 10,000$ and stays there for the remaining 40,000 steps. Whole-graph exact match does not follow it up: it sits at 0 throughout, with one exception — a transient spike to 0.22 around step 9,000 that collapses back to 0 a thousand steps later. That spike lands exactly where `cut(2, 3)` is still mid-climb and `scale/bias` are moving fastest; it is training noise at a crossing point, not a solved cell. On (7, 15, 24), `cut(2, 3)` only reaches a plateau around 0.50 and exact match never leaves 0. Both cells show the predicted-positive rate falling sharply as fine-tuning proceeds ($0.85 \rightarrow 0.27$ for (15, 15, 16), $0.76 \rightarrow 0.54$ for (7, 15, 24)): the two scalars are pushing the whole graph toward predicting fewer pairs connected overall, not selectively toward the one pair that was wrong.

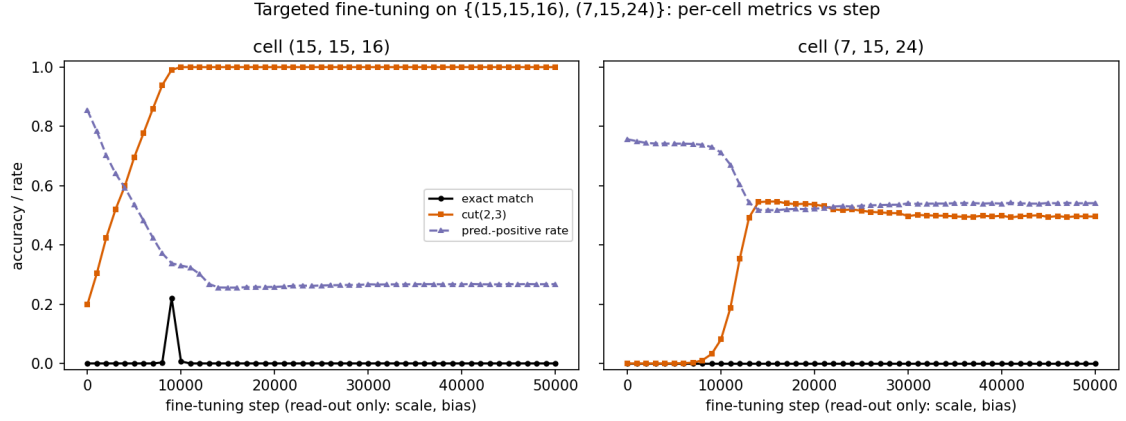


Figure 1: *Model*: split-chains-only, $n=46$, seed 1000, similarity read-out checkpoint of Report IX, fine-tuned as in §2.1. *Test set*: the two target cells, (15, 15, 16) and (7, 15, 24), 300 independently relabelled graphs per evaluation. *Metric*: whole-graph exact match, cut(2,3) (the pair that fails at step 0), and the overall predicted-positive rate, vs. fine-tuning step.

2.2.2 Does it learn past the doubled wall?

Figure 2 shows why the cut(2,3) improvement on (7,15,24) is not the win it looks like in isolation. Before fine-tuning, reach inside the 24-node component is ≈ 1 in all three distance buckets, including beyond the doubled wall $d > 18$ — consistent with Report IX’s finding that reach transfers even past $2 \cdot 3^L$. In the same window where cut(2,3) rises (steps $\sim 8,000$ – $14,000$), reach beyond the doubled wall collapses from ≈ 1.0 to ≈ 0.10 – 0.14 and never recovers; the within-capacity and between-the-walls buckets take a smaller hit, settling around 0.89–0.94. The two-scalar fine-tune does not learn past the doubled wall: it trades away almost all of the pre-existing long-range reach in the 24-node component for a partial fix to one cross-component cut, and even that trade only buys cut(2,3) up to ≈ 0.50 .

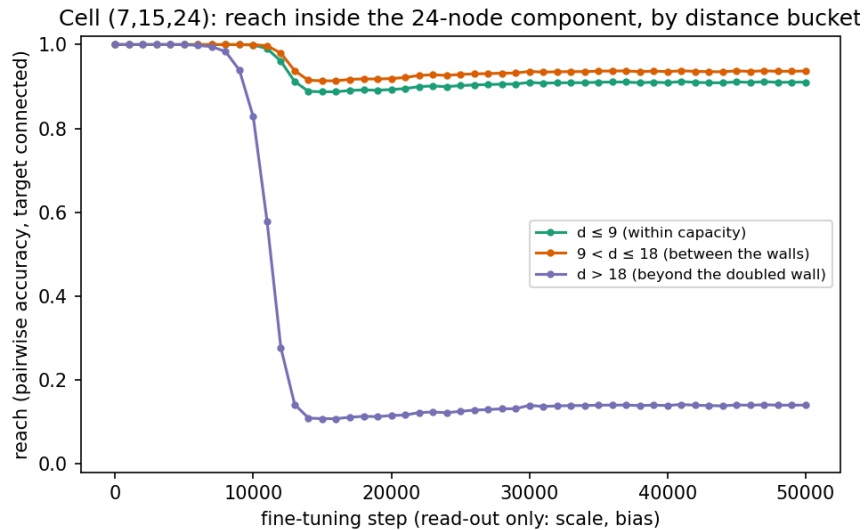


Figure 2: *Model/training as in Figure 1*. *Test set*: cell (7, 15, 24), 300 graphs per evaluation. *Metric*: pairwise reach (target connected) inside the 24-node component, split by shortest-path distance bucket, vs. fine-tuning step.

2.2.3 Does it forget the $K=2$ split-chains behaviour?

Figure 3 shows the cost on the behaviour the checkpoint already had. Before fine-tuning this is the clean step function from Report IX (the beyond-threshold table in `paper_draft.tex`, §5.1): exact match 1.000 for every $a \leq 11$, 0 from $a = 12$. After fine-tuning on $(15,15,16)/(7,15,24)$, the curve is worse in two distinct ways. First, a new failure appears well inside the previously-perfect region: $a = 2$ drops to 0 (recovering to 1.000 again at $a = 3,4$), even though $a = 2$ has nothing to do with either target cell. Second, the region $a = 5\text{--}11$, previously flat at 1.000, now degrades gradually (0.940, 0.887, 0.667, 0.567, 0.183, 0.333, 0.307) instead of holding to the wall. The per-split numbers behind this curve (`runs/report10/asym_chains_n46/n46_splitchains_seed1000_threeway_post/asym`) show cut staying at exactly 1.000 at *every single split*, unchanged from before fine-tuning: every new failure here is a reach story. At $a = 2$, reach of the long component drops to 0.190 while reach of the short component stays 1.000; from $a = 12$ on, reach of the long component collapses to 0.05–0.11 (it was 1.000 everywhere pre-fine-tuning). Fine-tuning has not just cost accuracy on the $K=2$ family, it has flipped which of reach/cut does the failing: the untouched checkpoint’s $K=2$ wall was a pure cut failure (reach 1.000 throughout, cut collapsing from $a = 12$); this checkpoint’s $K=2$ wall is now a reach failure with cut untouched.

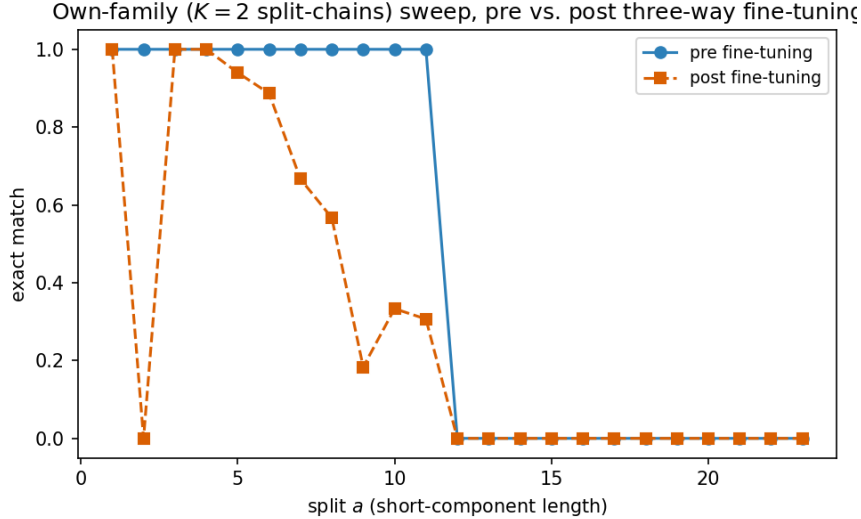


Figure 3: *Model*: the same checkpoint, before (Report IX, untouched) vs. after this fine-tuning. *Test set*: split-chain graphs at every split $a=1..23$, 300 graphs per split. *Metric*: whole-graph exact match vs. split a .

2.2.4 How much did scale/bias actually move?

Before fine-tuning: scale = 31.12, bias = -1.38 . After the 50,000-step targeted run: scale = 11.49 (-63%), bias = -7.87 . The pair of decision-boundary scalars moved a lot, in both directions at once: scale shrinks toward 0 (every cosine similarity counts for less), and bias moves sharply negative (the model needs stronger positive evidence before declaring two nodes connected). The post-fine-tuning bias is close to the -5 to -8 range typical of checkpoints trained on this distribution family from scratch (this checkpoint’s own pre-fine-tuning bias, -1.38 , was on the low/permissive end of that range); the fine-tune has moved the decision boundary toward a stricter operating point overall, not applied a small local correction. That fixing one specific miscalibrated cut required this large and global a shift is itself evidence against a pure threshold-calibration story: a small nudge would have sufficed if the two named cells’ problem were only that.

2.2.5 Mechanistic comparison: attention and the five-stage geometry, before vs. after

The post-fine-tuning stagewise battery (`runs/report10/stagewise_threeway/n46_splitchains_seed1000_t`) reused Report IX’s five probe cells, not just the two targets, so the untrained cells serve as a transfer check. Since the trunk is frozen, every stage up to $H^{(2)}$ (the final read-out geometry) is identical to Report IX’s — only the $H^{(2)}$ decision itself can move, because only scale/bias changed. Comparing $H^{(2)}$ cut(1,2)/cut(1,3)/cut(2,3) before (Report IX) vs. after this fine-tuning:

Cell	cut(1, 2) before→after	cut(1, 3) before→after	cut(2, 3) before→after
(1, 22, 23) – untrained	1.000 → 1.000	1.000 → 1.000	0.000 → 0.000
(4, 15, 27) – untrained	1.000 → 1.000	1.000 → 1.000	0.000 → 0.000
(2, 10, 34) – untrained	1.000 → 1.000	0.000 → 0.000	1.000 → 1.000
(7, 15, 24) – trained	0.999 → 1.000	0.923 → 1.000	0.000 → 0.458
(15, 15, 16) – trained	0.246 → 1.000	0.198 → 1.000	0.188 → 1.000

Table 1: $H^{(2)}$ pairwise cut, Report IX checkpoint vs. after Experiment 1’s fine-tuning. The three untrained cells are numerically identical before and after, to three decimals.

The three cells never included in the fine-tuning stream are unchanged to three decimals — including (1, 22, 23) and (4, 15, 27), whose cut(2, 3) was 0 before and stays exactly 0 after. This is the clearest single number in this experiment: fine-tuning on two specific cells moves those two cells and nothing else, at this stage.

Does that also hold across the whole $K=3$ grid, not just these five cells? Regenerating Report IX’s full 176-cell sweep (`plot_threeway_splitchains.py`) on the fine-tuned checkpoint gives a fuller and more surprising picture than the five named cells suggest. Figure 4 plots cut(1, 2)/cut(1, 3)/cut(2, 3) and exact match, averaged over every feasible s_2 at each s_1 , before vs. after. Before fine-tuning, cut(1, 2) and cut(1, 3) are moderate and noisy (0.5–0.95 across most of $s_1 \leq 11$), cut(2, 3) is close to 0 almost everywhere except an isolated spike to ≈ 1.0 at $s_1=2$, and exact match is 0 at every single cell. After fine-tuning on just two cells, cut(1, 2) jumps to ≈ 1.0 across the *entire* range $s_1=1$ –15, and cut(1, 3) turns from noisy into a clean rising trend from ≈ 0.65 up to 1.0 by $s_1=12$. cut(2, 3) stays the hardest pair — still a jagged function of s_1 with a wide dip through $s_1 \approx 6$ –9 — but it now reaches 1.0 by $s_1=12$, where before it never left 0.

Despite this broad, largely unintended improvement in the aggregate cut curves, whole-graph exact match barely moves: mean exact match over the 176 cells is 0.0100 after fine-tuning (a handful of cells reach as high as 0.87, but almost all sit at 0), essentially the same qualitative picture as the 0 everywhere before. Two scalars that make two of the three pairwise cuts look substantially more solved in aggregate still cannot produce an exact match, because (per §2.2.3) the same fine-tuning is simultaneously damaging reach — and exact match needs every pair right at once, cut and reach together.

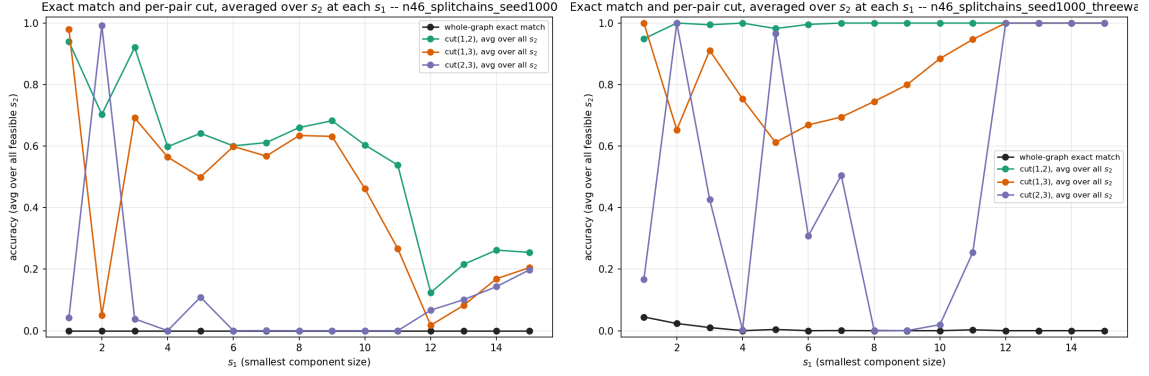


Figure 4: *Model*: split-chains-only, $n=46$, seed 1000 checkpoint, before (left, Report IX) vs. after (right) Experiment 1’s fine-tuning. *Test set*: every feasible three-way split-chain cell, $s_1=1..15$, averaged over all feasible s_2 at each s_1 , 176 cells total. *Metric*: whole-graph exact match and the three pairwise cuts.

2.3 Reading, once the data is in

(1) **Did it work on (15,15,16)?** Not in the sense that matters: cut(2,3) reaches 1.000, but exact match never leaves 0 (bar one transient noise spike) because reach on that same cell degrades from 1.000 to 0.73–0.82 across its three components. (2) **Did it transfer to (7,15,24), or did the two cells trade off?** Partial transfer, not a trade-off in the zero-sum sense — both cells’ cut(2,3) improve simultaneously (to 1.0 and ≈ 0.50 respectively) — but neither reaches exact match, and (7,15,24)’s partial gain costs it almost all of its long-range reach beyond the doubled wall. (3) **Did any learning happen on pairs beyond the doubled wall?** No: reach beyond $d=18$ in the 24-node component *regresses*, from ≈ 1.0 to ≈ 0.10 –0.14; nothing was learned past the wall, something that was already there was lost. (4) **Was $K=2$ behaviour preserved?** No. Cut stays perfect everywhere, but a new reach failure appears both at a small, previously-irrelevant split ($a=2$) and gradually across $a=5$ –11, where the checkpoint used to be exactly 1.000. (5) **Is the fix a real geometric change, or a pure threshold shift?** Given everything but scale/bias is frozen, the only thing that can move *is* the threshold by construction — but §2.2.5 shows this single shift has a much broader effect than the two named cells: cut(1,2) and cut(1,3) improve across nearly the whole $K=3$ grid, not just on the trained distribution, while cut(2,3) improves only partially and unevenly. That broad an effect from two scalars is itself informative: the geometry the trunk already produces evidently places most $K=3$ cross-component pairs close to the decision boundary, so a global bias/scale shift moves many of them at once — in both directions. **Net verdict for Experiment 1:** the targeted fine-tune is not a free threshold correction. It trades reach (own-family and beyond-the-wall) for a partial, uneven improvement in cut, and does not produce exact match on either target cell.

3 Experiment 2: fine-tuning on the full $K=3$ split distribution

Experiment 1 asks whether the two-scalar fine-tune can fix two specific cells it is trained directly on. It says nothing about whether that fix is a genuinely general three-component rule or an overfit to those two cells’ particular geometry. Experiment 2 asks the general-rule question directly: fine-tune the same two scalars, from the same starting checkpoint, but on *every* three-way split instead of two, for a much longer budget.

3.1 Setup

Checkpoint. Identical starting point to Experiment 1 (§2.1): the unmodified split-chains-only, $n=46$, seed-1000 checkpoint.

Fine-tuning. Every parameter frozen except `sim_scale/sim_bias`, as in Experiment 1. Training stream: `generate_path_union_graph(n, rng, min_paths=3, max_paths=3)` — exactly three components every sample, with both cut points (hence every component-size combination) drawn uniformly, freshly relabelled every sample — instead of only the two named cells. 200,000 steps ($4\times$ Experiment 1’s budget), batch 500, AdamW lr 10^{-3} (`finetune_readout_threeway_full.py`).

What is measured. The same per-cell metrics as Experiment 1 on the *same* two named cells (15, 15, 16)/(7, 15, 24) (for a direct, apples-to-apples comparison against Experiment 1’s curves, Figures 1–2), *plus* an aggregate exact-match/pairwise-accuracy number over freshly sampled, generic three-component graphs at every evaluation (the same style of number Report IX’s own generic- $K\geq 3$ fine-tuning probe reported). The own-family $K=2$ sweep and the mechanistic battery (attention, five-stage geometry) are reproduced exactly as in Experiment 1, into the `_full3way`-tagged output directories.

3.2 Results

3.2.1 Does exposure to the full distribution generalise, where the targeted fine-tune could only overfit two cells?

Figure 5 gives the direct answer: no. Aggregate exact match over freshly sampled, generic three-component graphs rises from 0.0 at step 0 to a peak of ≈ 0.05 around step 15,000–20,000, then settles back down to ≈ 0.01 –0.02 for the remaining 180,000 steps — a handful of graphs out of 300 per evaluation, not a general rule. Pairwise accuracy rises more substantially, from 0.62 to a plateau around 0.74, but pairwise accuracy is a much weaker bar than exact match (a graph with one wrong pair out of hundreds still scores highly on it). Four times the steps and exposure to the entire $K=3$ split space, instead of two named cells, does not turn read-out-only fine-tuning into a general three-component rule: it confirms, at a longer budget and a purer $K=3$ stream than Report IX’s own probe (`scratch_prof_experiments.tex`, §6, 3,000 steps on a mixed $K \in \{3, 4, 5, 6\}$ stream), that the representational gap sits in the frozen trunk, not in the read-out threshold.

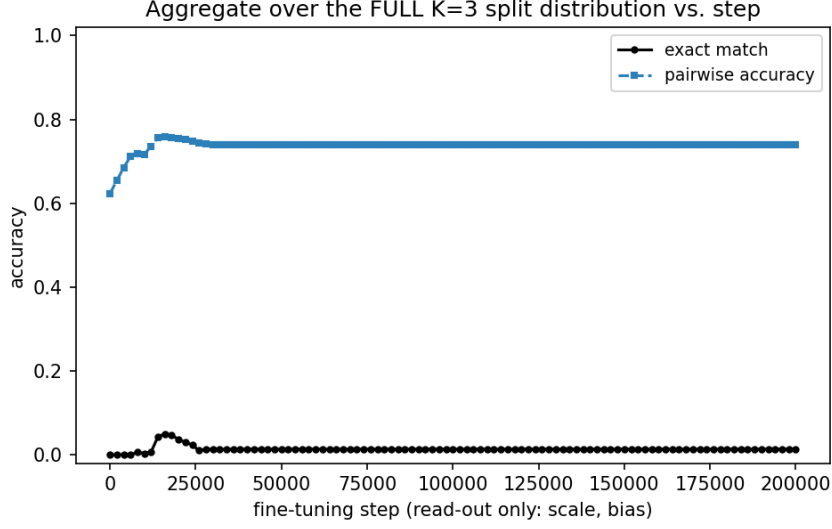


Figure 5: *Model*: split-chains-only, $n=46$, seed 1000, similarity read-out checkpoint of Report IX, fine-tuned as in §3.1. *Test set*: freshly sampled, generic three-component `path_union` graphs, 300 per evaluation. *Metric*: whole-graph exact match and pairwise accuracy, vs. fine-tuning step.

3.2.2 Same two named cells as Experiment 1 — does the broader stream do better, worse, or the same?

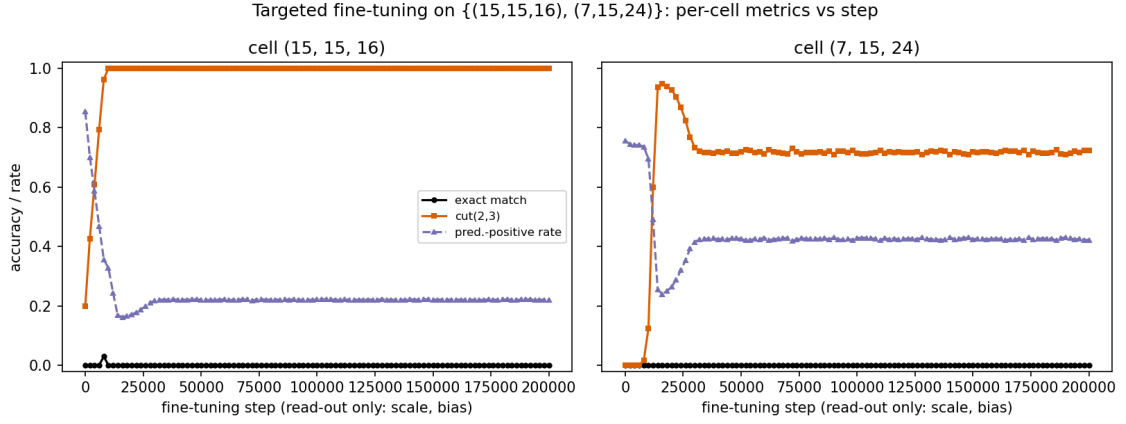


Figure 6: *Model*: same starting checkpoint as Figure 1, fine-tuned as in §3.1 (full $K=3$ distribution, 200,000 steps). *Test set*: the same two named cells, (15, 15, 16) and (7, 15, 24). *Metric*: as in Figure 1.

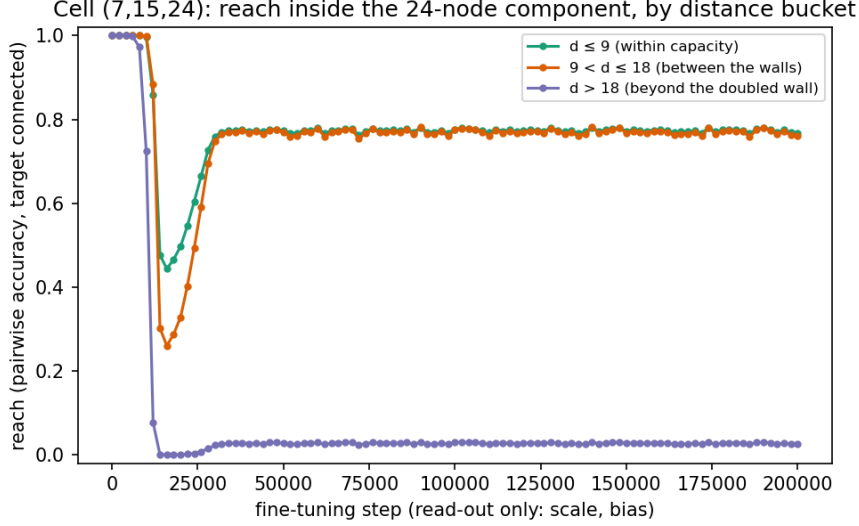


Figure 7: *Model/training as in Figure 6. Test set/metric:* as in Figure 2.

On the two named cells, the broader stream does slightly better on $(7, 15, 24)$'s $\text{cut}(2, 3)$ (0.72 at step 200,000 vs. Experiment 1's 0.50) but slightly *worse* everywhere else. $(15, 15, 16)$'s reach degrades further (components at 0.57–0.67 vs. Experiment 1's 0.73–0.82; between-the-walls reach on its own down to 0.13 vs. 0.29). $(7, 15, 24)$'s reach beyond the doubled wall is essentially fully destroyed either way (0.025 here vs. 0.14 for the targeted run — both near-zero). Exact match on both named cells stays 0 in both experiments. The pattern from §2.2.5 repeats at the level of the full grid: $\text{cut}(1, 2)/\text{cut}(1, 3)$ again jump to ≈ 1.0 almost everywhere and $\text{cut}(2, 3)$ again follows the same jagged shape with a dip through $s_1 \approx 6\text{--}9$, but mean exact match over the 176 cells is 0.0174 — marginally higher than Experiment 1's 0.0100, still nowhere near a general rule. Longer, broader exposure does not change the qualitative story, only nudges a few numbers in either direction.

3.2.3 Own-family retention and scale/bias movement

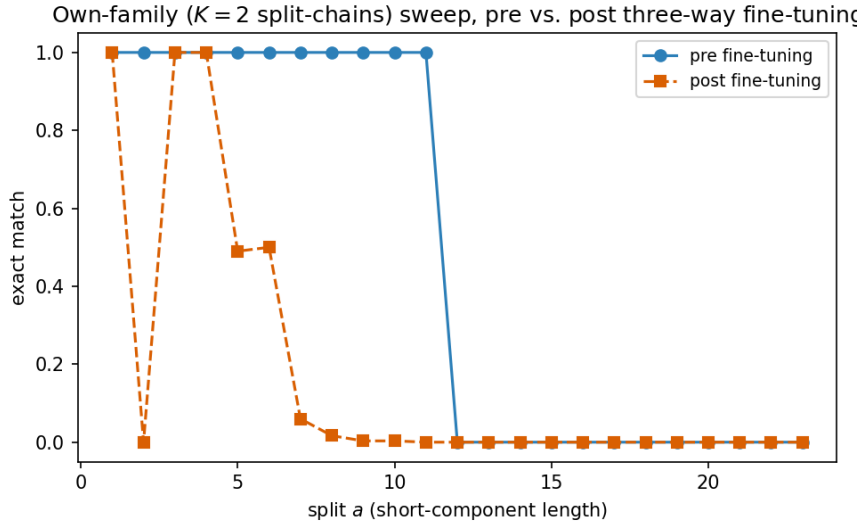


Figure 8: *Model:* the same checkpoint, before (Report IX, untouched) vs. after the full-distribution fine-tuning of §3.1. *Test set/metric:* as in Figure 3.

Scale/bias: before = (31.12, -1.38), after 200,000 full-distribution steps = (4.83, -3.59) — scale shrinks even further than Experiment 1’s (-84% vs. -63%), while bias ends up *less* negative than Experiment 1’s (-3.59 vs. -7.87). The own-family sweep is damaged in the same qualitative way as Experiment 1 (new failure at $a=2$, gradual decay through $a=5$ –11, cut untouched at 1.000 throughout), but more severely in the middle of the range: exact match at $a=7$ –10 drops to 0.06, 0.02, 0.003, 0.003 (vs. Experiment 1’s 0.667, 0.567, 0.183, 0.333), and reach_long at $a=11$ already dips to 0.925 where Experiment 1 still held 0.978. Four times the fine-tuning budget does not stabilize the own-family behaviour; it erodes it somewhat further.

3.3 Reading, once the data is in

The aggregate $K=3$ exact match does not move meaningfully above 0 (0.013 at 200,000 steps, after a small transient peak of 0.05): read-out-only fine-tuning does not produce a genuinely general three-component rule, no matter how much coverage or how long the budget. This confirms, at much higher confidence than Report IX’s shorter, mixed- K probe (§6 of `scratch_prof_experiments.tex`, 3,000 steps), that the representational gap is in the frozen trunk, not the read-out threshold. Comparing directly against Experiment 1: the full-distribution run does not do uniformly worse or better on the two named cells — it trades a slightly better cut(2, 3) on (7, 15, 24) for slightly worse reach everywhere else, and a somewhat more damaged own-family sweep in the middle range. Spreading two scalars’ worth of capacity over the whole $K=3$ space does not cost the two hardest cells much relative to the targeted run, because neither run was ever going to reach exact match on them: both are re-arranging the same fixed, frozen geometry along the one axis (scale/bias) available to them, and that axis is not expressive enough to fix more than a fraction of the cross-component pairs at once.

4 Combined verdict

No amount of read-out-only fine-tuning tested here — targeted on two cells for 50k steps, or exposed to the entire $K=3$ split space for 200k steps — gets a three-component split-chains graph exactly right. Both runs move the aggregate pairwise cuts substantially (cut(1, 2) and cut(1, 3) go from noisy/moderate to ≈ 1.0 across nearly the whole grid; cut(2, 3) improves unevenly), which on first glance looks like real progress on §5.3’s question. But whole-graph exact match stays at ≈ 0 –0.02 throughout, because the same fine-tuning damages reach: on the very $K=2$ split-chains family the checkpoint used to solve perfectly, exact match now degrades gradually from $a=5$ and fails outright at a small split ($a=2$) that was never a problem before, and reach beyond the doubled wall in the 24-node component of (7, 15, 24) collapses from ≈ 1 to ≈ 0.03 –0.14. The two read-out scalars are expressive enough to move a decision boundary that affects most pairs in the graph at once, but not expressive enough to separate “pairs that need a stricter threshold” (the failing cross-component cuts) from “pairs that need the old, looser threshold” (long-range within-component reach) — because both kinds of pairs sit in the same region of the cosine-similarity geometry the frozen trunk already produces. **For §5.3: the fine-tuning experiments support describing the $K \geq 3$ failure as a genuine representational gap in the trunk, not a miscalibrated read-out threshold — and additionally show that trying to patch the threshold anyway has a real cost, silently breaking $K=2$ behaviour that was previously perfect.** This is consistent with, and now more thoroughly evidenced than, Report IX’s own generic fine-tuning probe (`scratch_prof_experiments.tex`, §6, experiment 6), which reached the same conclusion at a shorter budget and a narrower training stream.

5 Extending the four main paper experiments to 5 (two-path) / 10 (two-cycle) seeds — and applying the same treatment to cycles

This section is analysis for Edoardo to review, not paper text. The two-path tables/figures below are the direct 5-seed update of `paper_draft.tex` §5.1 and its full-distribution-control appendix. The two-cycle tables/figures are new — built with the identical method and style, since §5.2 “Cycles graphs” is still an empty placeholder in `paper_draft.tex` and nothing like this has been produced for cycles before — and, following the seed-lottery variability the first 5 cycle seeds showed, extended straight to 10 seeds per `istruzioni.md`’s policy for high-variance conditions.

All four tables use the same columns as `paper_draft.tex`’s Table 1 (test split, exact match, reach, cut; mean $\pm$ std over seeds, consecutive rows grouped only where all three values match to three decimals). All four figures use the same style as `paper_figures/fig_oddtrain_exactmatch_vs_a.pdf`: thick blue line = mean over seeds, thin gray lines = individual seeds, filled/hollow markers for split present/never-present in training where the condition has a training grid, dashed vertical line at the empirical break with an honest label (never “capacity boundary”). Source: `plot_report10_fiveseed.py` (new script, added for this section; reads `runs/report9/asm_chains_n46/*/asm_chains.json` for the two chain conditions and `runs/report9/mechanistic/*/metrics.csv` for the two cycle conditions, since `eval_asym_chains.py` is chain-only and the cycle sweep was only ever produced via `mechanistic_asym_chains.py -topology cycle`, in the long/tidy CSV format).

5.1 Two-path, odd/grid training (5 seeds) — direct update of Table 1 / Figure 1

Test split (a , $46 - a$)	Exact match	Reach	Cut
(1, 45)	0.301 ± 0.275	1.000	0.442 ± 0.339
(2, 44) – (10, 36)	1.000	1.000	1.000
(11, 35)	0.999 ± 0.001	1.000	1.000
(12, 34)	0.000	1.000	0.469 ± 0.026
(13, 33)	0.000	1.000	0.019 ± 0.009
(14, 32) – (23, 23)	0.000	1.000	0.000

Table 2: Two-path model trained on the odd setting ($n=46$, similarity read-out, 5 seeds). Exact match, reach, and cut by test split.

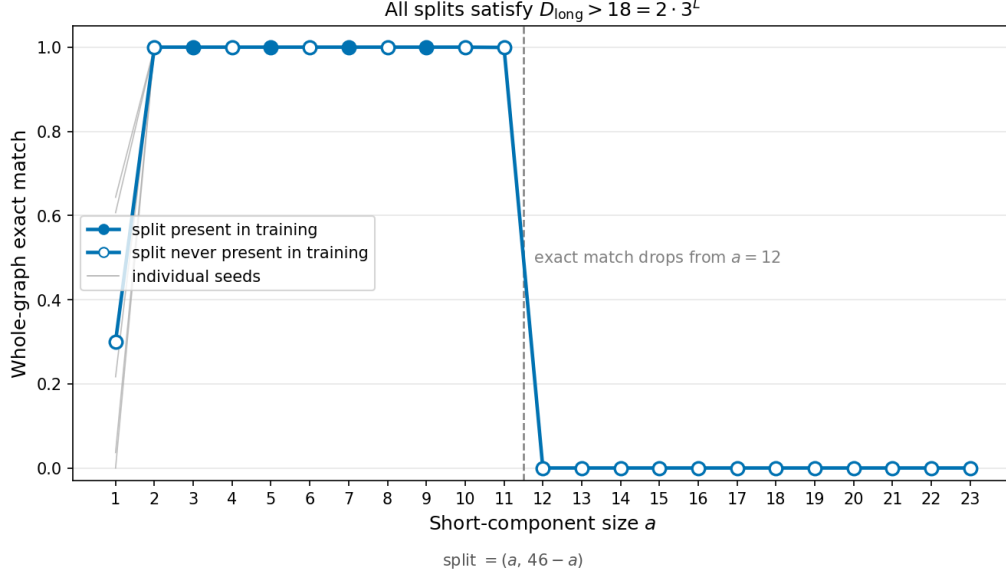


Figure 9: *Model*: two-path, odd setting (training on $a \in \{3, 5, 7, 9\}$ only), $n=46$, similarity read-out, 5 seeds. *Test set*: every split $a=1..23$, 300 graphs per split per seed. *Metric*: whole-graph exact match.

Extending from 2 to 5 seeds changes nothing about the story already in `paper_draft.tex`: exact match 1.000 for $a=2-10$ (seen or not), 0.999 at $a=11$, breaking at $a=12$, cut collapsing to 0 by $a=14$. The two new seeds mostly tighten the error bars at the break ($a=12$ cut: 0.469 ± 0.026 vs. the 2-seed 0.465 ± 0.009 ; $a=13$: 0.019 ± 0.009 vs. 0.028 ± 0.002) rather than changing the picture. The one genuinely new data point is $a=1$ (a single isolated node as the short component, outside `paper_draft.tex`'s reported range which starts at $a=2$): here exact match is 0.301 ± 0.275 and cut 0.442 ± 0.339 — wildly seed-dependent even though $a=1$ is deep inside the trained-adjacent region and has nothing to do with the $a=12$ wall. This is visible directly in Figure 9 as the individual seed lines fanning out only at $a=1$. Consistent with the isolated-node fragility already documented in `scratch_prof_experiments.tex` §5 (Report IX follow-up experiment 5), now seen here as a seed-lottery-flavoured instability rather than a uniform failure.

5.2 Two-path, full split-size distribution (5 seeds) — direct update of the full-distribution-control appendix

Test split $(a, 46 - a)$	Exact match	Reach	Cut
$(1, 45) - (10, 36)$	1.000	1.000	1.000
$(11, 35)$	0.990 ± 0.020	0.998 ± 0.005	1.000
$(12, 34)$	0.000	0.786 ± 0.001	1.000
$(13, 33)$	0.000	0.800 ± 0.002	1.000
$(14, 32)$	0.000	0.816 ± 0.001	1.000
$(15, 31)$	0.000	0.831 ± 0.002	1.000
$(16, 30)$	0.000	0.848 ± 0.001	1.000
$(17, 29)$	0.000	0.864 ± 0.001	1.000
$(18, 28)$	0.000	0.880 ± 0.001	1.000
$(19, 27)$	0.000	0.897 ± 0.001	1.000
$(20, 26)$	0.000	0.913 ± 0.001	1.000
$(21, 25)$	0.000	0.929 ± 0.001	1.000
$(22, 24)$	0.000	0.945 ± 0.001	1.000
$(23, 23)$	0.000	0.960 ± 0.001	1.000

Table 3: Two-path model trained on the full split-size distribution ($n=46$, similarity read-out, 5 seeds). Exact match, reach, and cut by test split.

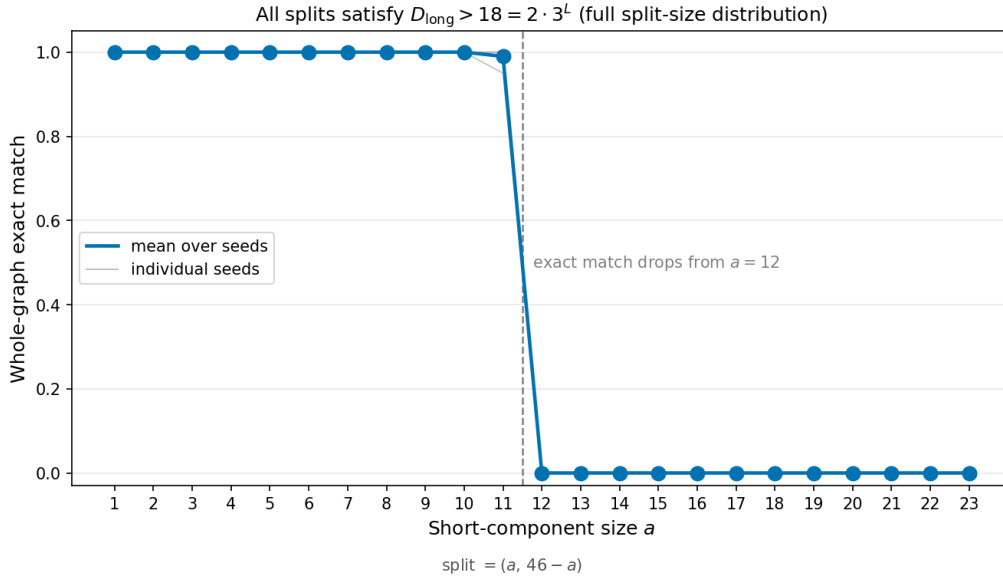


Figure 10: *Model*: two-path, full split-size distribution training (a uniform over $1..45$), $n=46$, similarity read-out, 5 seeds. *Test set/metric*: as in Figure 9. No filled/hollow marker distinction: every split is in-distribution here.

The 5th seed changes essentially nothing: identical break at $a=12$, reach recovering gradually from 0.786 to 0.960 exactly as in the 4-seed appendix table, error bars uniformly tiny (≤ 0.02 everywhere). This condition is fully stable across seeds, unlike either cycle condition below.

5.3 Two-cycle, odd/grid training (10 seeds) — new, not yet in the paper

Test split ($a, 46 - a$)	Exact match	Reach	Cut
(3, 43)	0.895 ± 0.298	1.000	0.900 ± 0.300
(4, 42)	0.773 ± 0.385	1.000	0.814 ± 0.345
(5, 41)	0.788 ± 0.392	1.000	0.832 ± 0.340
(6, 40)	0.544 ± 0.323	0.994 ± 0.018	0.776 ± 0.236
(7, 39)	0.841 ± 0.319	0.995 ± 0.015	0.877 ± 0.247
(8, 38)	0.840 ± 0.326	0.996 ± 0.013	0.858 ± 0.294
(9, 37)	0.802 ± 0.395	0.993 ± 0.020	0.850 ± 0.310
(10, 36)	0.748 ± 0.388	0.991 ± 0.028	0.834 ± 0.329
(11, 35)	0.740 ± 0.388	0.991 ± 0.026	0.799 ± 0.348
(12, 34)	0.190 ± 0.316	0.992 ± 0.025	0.445 ± 0.407
(13, 33)	0.000	0.991 ± 0.026	0.014 ± 0.040
(14, 32)	0.000	0.991 ± 0.027	0.013 ± 0.040
(15, 31)	0.000	0.992 ± 0.025	0.013 ± 0.038
(16, 30)	0.000	0.991 ± 0.027	0.012 ± 0.036
(17, 29)	0.000	0.992 ± 0.023	0.013 ± 0.038
(18, 28)	0.000	0.992 ± 0.024	0.012 ± 0.036
(19, 27)	0.000	0.993 ± 0.019	0.012 ± 0.036
(20, 26)	0.000	0.993 ± 0.020	0.012 ± 0.036
(21, 25)	0.000	0.993 ± 0.022	0.012 ± 0.037
(22, 24)	0.000	0.995 ± 0.015	0.013 ± 0.039
(23, 23)	0.000	0.993 ± 0.022	0.013 ± 0.039

Table 4: Two-cycle model trained on the odd/grid setting ($n=46$, similarity read-out, 10 seeds). Exact match, reach, and cut by test split.

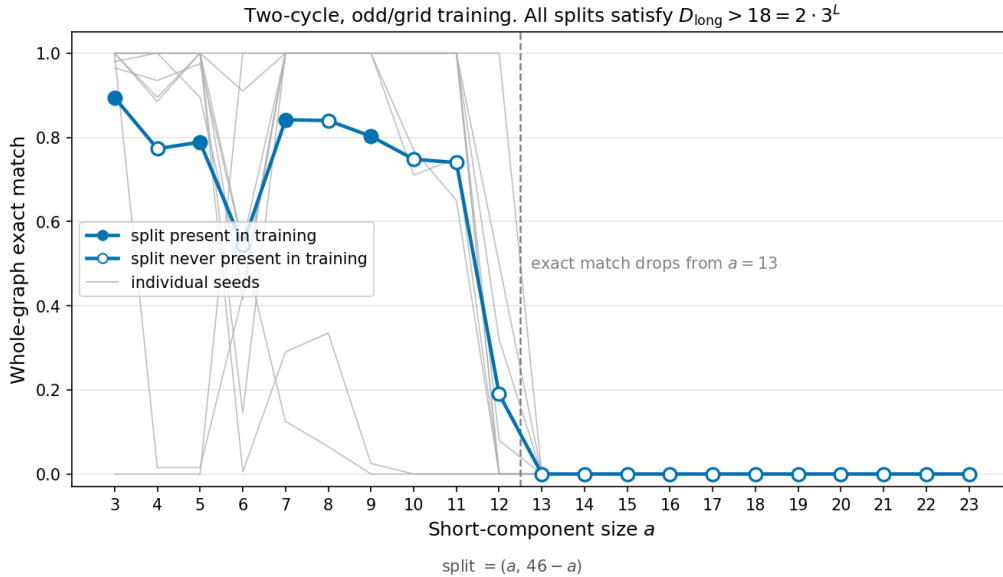


Figure 11: *Model*: two-cycle, odd/grid setting (training on $a \in \{3, 5, 7, 9\}$ only), $n=46$, similarity read-out, 10 seeds. *Test set/metric*: as in Figure 9.

Qualitatively different from the two-path conditions in every respect. There is no clean plateau-then-cliff: exact match already sags within the trained sizes $a=3-5$, drops further and unevenly through $a=6-12$ (individual seeds visibly disagree on the shape, not just the level — see the crossing lines in Figure 11), and almost every seed reaches a clean 0 from $a=13$. Extending

from 5 to 10 seeds adds one genuinely new pattern rather than just more of the same: cut at $a=12-23$, per seed — 1000=0.00 (throughout), 2000=0.92 $\rightarrow$ 0.00 (from $a=13$), 3000=0.00, 4000=0.00, 5000=**0.14** $\rightarrow$ **0.12–0.13** (*stays there for the rest of the sweep*), 6000=0.77 $\rightarrow$ 0.00, 7000=0.17 $\rightarrow$ 0.00, 8000=0.48 $\rightarrow$ 0.00, 9000=1.00 $\rightarrow$ 0.00 (perfect exactly at the break, then a cliff one step later), 10000=0.95 $\rightarrow$ 0.00. **No seed reaches a sustained solve, but seed 5000 is the only one that settles into a low, stable, non-zero plateau ($\approx 0.12-0.14$) instead of collapsing to exactly 0** — a third outcome distinct from both “solves” and “collapses completely,” visible only because the other five seeds gave a wide enough sample to see it is not the norm. At the fully balanced split $a=23$: only seed 5000 is above 0 (0.13); every other seed is exactly 0.000.

5.4 Two-cycle, full split-size distribution (10 seeds) — new, not yet in the paper

Test split ($a, 46 - a$)	Exact match	Reach	Cut
(3, 43)	0.731 ± 0.247	0.999 ± 0.001	0.995 ± 0.005
(4, 42)	0.642 ± 0.301	1.000	0.950 ± 0.050
(5, 41)	0.629 ± 0.333	1.000	0.913 ± 0.100
(6, 40)	0.898 ± 0.299	0.962 ± 0.115	1.000
(7, 39)	0.648 ± 0.433	0.940 ± 0.108	1.000
(8, 38)	0.665 ± 0.415	0.949 ± 0.102	1.000
(9, 37)	0.657 ± 0.421	0.957 ± 0.097	1.000
(10, 36)	0.670 ± 0.402	0.965 ± 0.092	1.000
(11, 35)	0.759 ± 0.298	0.971 ± 0.088	0.996 ± 0.011
(12, 34)	0.295 ± 0.451	0.973 ± 0.081	0.400 ± 0.490
(13, 33)	0.300 ± 0.458	0.975 ± 0.074	0.400 ± 0.489
(14, 32)	0.298 ± 0.455	0.978 ± 0.067	0.399 ± 0.489
(15, 31)	0.295 ± 0.451	0.980 ± 0.059	0.399 ± 0.489
(16, 30)	0.271 ± 0.416	0.983 ± 0.051	0.399 ± 0.489
(17, 29)	0.225 ± 0.353	0.986 ± 0.043	0.398 ± 0.488
(18, 28)	0.289 ± 0.443	0.989 ± 0.033	0.399 ± 0.489
(19, 27)	0.273 ± 0.417	0.992 ± 0.023	0.399 ± 0.489
(20, 26)	0.272 ± 0.418	0.996 ± 0.012	0.399 ± 0.489
(21, 25)	0.294 ± 0.404	1.000	0.399 ± 0.489
(22, 24)	0.293 ± 0.403	1.000	0.399 ± 0.489
(23, 23)	0.288 ± 0.405	1.000	0.399 ± 0.489

Table 5: Two-cycle model trained on the full split-size distribution ($n=46$, similarity read-out, 10 seeds). Exact match, reach, and cut by test split.

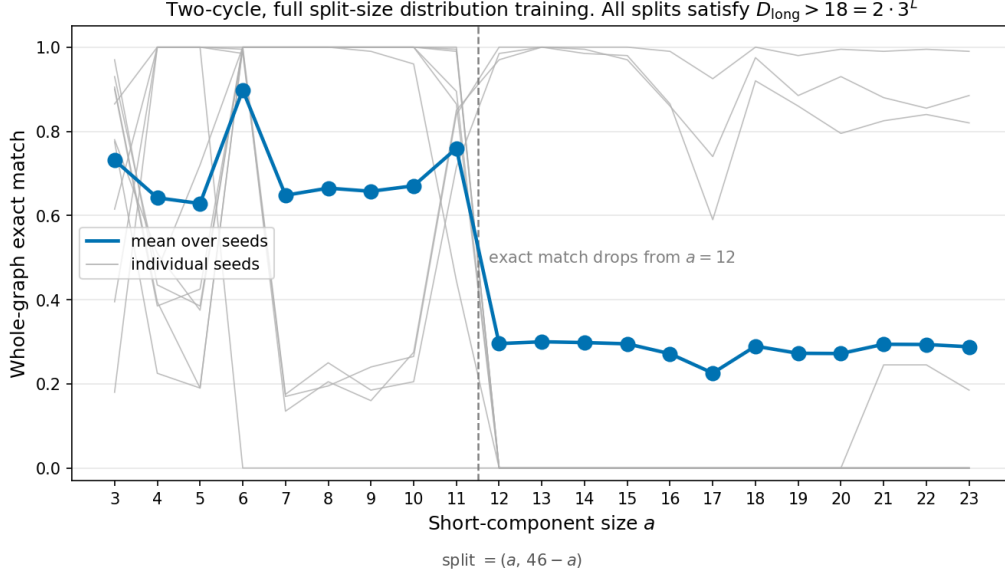


Figure 12: *Model*: two-cycle, full split-size distribution training, $n=46$, similarity read-out, 10 seeds. *Test set/metric*: as in Figure 10.

The mean column in this table is misleading on its own and should not be quoted without the per-seed breakdown next to it. From $a=12$ on, mean exact match and mean cut sit well below 1 with a large std — not because every seed gets a middling fraction of graphs right, but because the seeds split into qualitatively different groups. Cut at $a=12$ –23, per seed: 1000=0.00, 2000=0.00, 3000=1.00, 4000=0.00, 5000=1.00, 6000=0.00, 7000=0.00, 8000=0.00, 9000=0.99–1.00, 10000=0.99–1.00 — **6 of 10 seeds collapse to “everything connected” at every split from $a=12$ on; 4 of 10 solve the cut cleanly.** That 4/10 matches the 2/5 estimate from the first 5 seeds almost exactly (seeds 6000–8000 collapse, 9000 and 10000 join 3000/5000 in solving), so **the solving fraction itself looks stable at $\approx 40\%$ across both seed batches** — the earlier caution about “1/4 vs. 2/5” being different claims (previous version of this section) undersold how close the two independent 5-seed estimates actually turned out to be.

What extending to 10 seeds actually changes is qualitative, not just the count: **seed 9000 solves cut almost immediately (0.99–1.00 from $a=12$) but its reach inside the long component starts badly broken (0.729 at $a=12$) and recovers gradually and monotonically as the split balances (0.729, 0.752, 0.776, 0.802, 0.829, 0.858, 0.890, 0.924, 0.961, then 1.000 from $a=21$ on)** — exact match for this seed is exactly 0 through $a=20$ purely because of this reach gap, then jumps to 0.245/0.245/0.185 at $a=21, 22, 23$ once reach finally catches up. **This is the “some seeds grow after $a=12$ ” pattern noted before downloading these seeds**, and it is a real, distinct third regime: not the flat collapse of the 6 failing seeds, not the near-perfect immediate solve of seeds 3000/5000/10000 (whose own exact match is noisy but stays high, driven by small residual cut/reach jitter rather than a genuine gap that closes over the sweep), but a gradual-recovery shape identical in kind to the two-path *continuous* condition’s own reach recovery (§5.2, reach 0.786 $\rightarrow$ 0.960) — the first time this specific failure/recovery shape has shown up on a cycle rather than a chain. Figure 12 shows all of this at once: past $a=12$ the individual seed lines split into a group pinned near 1.0, a group pinned at exactly 0.0, and (less visually obvious among the noise, but confirmed in the per-seed numbers above) seed 9000 climbing steadily from the bottom group into the top one as $a \rightarrow 23$.

Grid vs. continuous training, for cycles specifically. The two cycle conditions are still not just noisier versions of each other now that both have more seeds: continuous training gives

close to even odds of landing a fully-solving seed at the hardest split (4/10, plus one seed — 9000 — that gets there eventually via gradual reach recovery), while grid training gives none (0/10 sustained solves; one seed, 5000, reaches only a stable partial plateau ≈ 0.12 – 0.14 , never above it). This still suggests the seed lottery on cycles interacts with *how* the training distribution is built (dense/continuous vs. sparse/grid), not only with initialization — a comparison that was not possible before, since §2 of `scratch_prof_experiments.tex` only had 2 seeds for the grid condition and reported seed disagreement without being able to say anything about the continuous condition’s own seed count at the time.

6 K=3 path-union training at a new canvas, n=60

This section is analysis for Edoardo to review, not paper text. Feeds the multipath generalisation question, not §5.3 directly.

New family `path_union_k3` added to `experiments2/train_families_n20.py` (`generate_path_union_graphs`, `max_paths=3`, `min_paths=3`) as a full TRAINING distribution, not only a read-out-only fine-tuning stream as in `finetune_readout_threeway_full.py`. Trained at $n=60$ (never used before in this project), seeds 1000/2000, otherwise identical setup to every other training in this report (RoBERTa, similarity read-out, normalized-ReLU attention, 10^6 steps, batch 1000). Both seeds COMPLETED 2026-08-26 in ≈ 4.5 h (well under the 30h budgeted — no historical timing existed yet for $n=60$, see `scripts/r10_n60_pathunion_k3_seed{1000,2000}.sbatch`). In-distribution validation exact match plateaus low for both seeds (best 0.194/0.195, final 0.187/0.187) — even on its own training distribution this checkpoint rarely gets every pair in a $K=3$ graph right at once, consistent with the $K \geq 3$ representational-gap reading from §4 above, now seen at initial training rather than only after read-out fine-tuning of an already-frozen $K=2$ trunk.

6.1 Exact match for every 3-way split-size combination

[PENDING — eval scripts written and smoke-tested, not yet run (checkpoints exist only on HPC): `scripts/r10_n60_pathunion_k3_eval_seed{1000,2000}.sbatch` runs `eval_threeway_splitchains.py` (already generic on n /checkpoint, the same tool used for the $n=46$ full-grid check in §2.2.5) over every partition of 60 into 3 parts. Once pulled, regenerate with `plot_threeway_splitchains.py -tag n60_pathunion_k3_seed{1000,2000} -report_root report10` and fill in this subsection. Planned reading, per Edoardo (2026-08-26): on the fully balanced cell (20,20,20), expect the opposite failure direction from the two-path balanced case (which over-connects/fails on cut with reach staying perfect) — here, since no component is short enough to anchor an endpoint-completion signal for the others, the prediction is under-connection: each path’s own two endpoints (nodes 1/20, 21/40, 41/60) called disconnected from each other, i.e. a within-component long-range reach failure on all three components symmetrically, not a cross-component cut failure.]

6.2 Multipath test: does parallelism still rescue an unseen shared-endpoint graph?

[PENDING — new script `mechanistic_multipath_geometry.py` (smoke-tested locally on a random-weight $n=60$ checkpoint, not yet run on the real checkpoints), invoked by the same `scripts/r10_n60_pathunion_k3_eval_seed{1000,2000}.sbatch`. Tests, on `generate_multipath_graph` graphs (two terminals s, t joined by $k \in \{1, 2, 3\}$ parallel routes of 19 edges each, terminal degree fixed at 4 regardless of k , leaf padding + sparse filler for the rest of the canvas) — a structure this checkpoint never saw in training (only disjoint 3-way splits, no shared endpoints, no leaves/filler, max degree 2): (1) `term_connect`, whether $s-t$

is predicted connected, for $k=1,2,3$ — the direct test of whether more parallel routes rescue the connection the way they did for every path_union-trained checkpoint since Report VI; (2) the final-stage ($H^{(2)}$, i.e. after the last layer’s MLP) cosine-similarity geometry, block-averaged by node-pair category (the s - t pair itself, within-route pairs, route-vs-route pairs, terminal-to-leaf, leaf-to-leaf, active-to-filler, within-filler), each converted through the checkpoint’s own scale/bias into an implied logit and connected/disconnected call — to see whether the geometry treats s - t as similar even in the k where the raw decision is wrong (cf. Report X Experiment 1’s own reading, §2.3, question 5, on how much of the residual error the geometry itself already carries). Fill in this subsection once pulled.]